

Lecture #14: practice Data Filtering / deduplication mechanics

Last lecture: overview dataset for training LLM.

- ①: live service (github): $\rightarrow$ dump/crawl (GIT Archive) $\rightarrow$ processed data (the toxicity stack)
- ②: processing: HTML to text, language, quality

This lecture: dive deep into mechanics.

- ①: Algorithms for filtering (e.g. classifiers)
- ②: Applications of filtering (e.g. language, quality, toxicity)
- ③: Deduplication (e.g. Bloom Filters, MinHash, LSH)

def filtering_algorithm:

- ①: Given some target data T and lots of raw data R (e.g. common crawl), find the subset T' from R , similar to target data T .

updates for filtering algorithms:

- #1: Generalize from the target data (want T and T' to be different)
- #2: Extremely fast (have to run it on R , which is huge)

#1: KenLM-main(): train a n -gram model

n -gram model with Kneser-Ney smoothing

- KenLM: fast implementation originally for machine translation
- common language model used for data filtering
- Extremely simple / fast - just count / normalize

$n=3: p(\text{in}/\text{the cat}) = \text{count}(\text{the cat in}) / \text{count}(\text{the cat})$
 $\#2: p(\text{in}/\text{the cat})$ depends on $p(\text{in}/\text{cat})$ too (cat)

n -gram language models: a statistical NLP model to predict probability of the next word in a sequence, based on the $n-1$ words that come before it, using Markov assumption

struggles with:
①: unseen sequences (data sparsity), sparse counts
②: long dependency

Solution: use Kneser-Ney smoothing to handle unseen n -grams

CCNet: used in LiMa: [Wenzek + 2019]. keep top 1/3

Summary: Kneser-Ney n -gram language models (with KenLM implementation), is fast but crude.

②. fastText-main(): # Train a classifier.

fastText-classifier [Joulin + 2016]

①: Task: text classification (e.g. sentiment classification)

②: Goal was to train a fast text classifier

③: good as slower neural network classifier.

Baseline: bag of words (not of what they did)

problem: $V \times K$ params could be huge.

fastText classifier: bag of word embeddings

only $H \times (V+K)$ parameters

implement: ①: parallelized, asynchronous SGD

②: learning rate: interpolation from [some number] to 0